

boba_extension_dim08

Dimension 08: Web Page DOM Scraping & Dynamic Content Detection

Research Objective

Comprehensive strategies for finding torrent and magnet links on any web page, including handling dynamically loaded content in SPAs, iframes, infinite scroll, and Shadow DOM scenarios.

Table of Contents

1. [Content Script Injection Timing](#)
 2. [DOM Traversal Strategies](#)
 3. [Link Element Detection Patterns](#)
 4. [Text-Based Magnet Detection](#)
 5. [Magnet Link URI Patterns](#)
 6. [.torrent File Link Patterns](#)
 7. [MutationObserver for Dynamic Content](#)
 8. [iframe Handling](#)
 9. [Shadow DOM Traversal](#)
 10. [Site-Specific Detection Patterns](#)
 11. [Performance Optimization](#)
 12. [Complete Implementation](#)
 13. [Edge Cases and Error Handling](#)
 14. [Test Cases](#)
 15. [Citations and Sources](#)
-

1. Content Script Injection Timing

1.1 Three Timing Options

Content scripts support three `run_at` values that determine when JavaScript is injected:

Value	Description	Use Case
<code>document_start</code>	DOM is still loading, before any other scripts run	Early setup, must wait for DOM manually
<code>document_end</code>	DOM is complete, but subresources still loading	Balance between early access and DOM readiness
<code>document_idle</code>	DOM and resources fully loaded (default)	Most content scripts, guaranteed DOM ready

Claim: The default `document_idle` is the recommended timing for most extensions. **Source:** Chrome for Developers - Content Scripts **URL:** <https://developer.chrome.com/docs/extensions/develop/concepts/content-scripts> **Date:** 2012-09-17 **Excerpt:** "document_idle" ... The browser chooses a time to inject scripts between "document_end" and immediately after the window.onload event fires... Use "document_idle" whenever possible. **Confidence:** high

1.2 Handling Different readyState

```
/**
 * Robust initialization that works regardless of when content
 * script loads.
 * Handles all three run_at scenarios: document_start,
 * document_end, document_idle
 */
function initializeWhenReady(callback) {
  if (document.readyState === 'loading') {
    // DOM still loading - wait for complete
    document.addEventListener('DOMContentLoaded', callback);
  } else if (document.readyState === 'interactive' ||
    document.readyState === 'complete') {
    // DOM ready - can run immediately
    callback();
  }
}

// Usage
initializeWhenReady(() => {
  performInitialScan();
});
```

```
    setupMutationObserver();  
});
```

1.3 Manifest Configuration

```
{  
  "content_scripts": [  
    {  
      "matches": ["<all_urls>"],  
      "js": ["content.js"],  
      "run_at": "document_idle",  
      "all_frames": false  
    }  
  ]  
}
```

1.4 Handling Dynamic SPAs (like Gmail)

Claim: For heavily dynamic SPAs like Gmail, `document_idle` may not be sufficient because the DOM continues to change after `window.onload`. **Source:** GMass Blog - Gmail Extension Timing **URL:** <https://www.gmass.co/blog/timing-gmail-chrome-extension-content-script/> **Date:** 2020-12-12 **Excerpt:** The confusion for me... is that the rules for Gmail are different... As long as you're wrapped in a `window.onload` if/then checking system, you're good. **Confidence:** high

```
/**  
 * SPA-safe initialization that handles both immediate and  
 * deferred DOM readiness.  
 * Essential for Gmail, Google Docs, and other heavy SPAs.  
 */  
function safeInitialize(callback) {  
  if (document.readyState === 'complete') {  
    callback();  
  } else {  
    window.addEventListener('load', callback);  
  }  
}
```

2. DOM Traversal Strategies

2.1 Basic `querySelectorAll` Approach

The most efficient method for finding link elements with torrent/magnet content:

```

/**
 * Fast initial scan using CSS selectors - O(1) for selector
 * matching.
 * This is the most performant approach for finding <a> tags.
 */
function findLinksBySelectors() {
  const results = [];

  // Magnet links in href attributes
  const magnetLinks =
    document.querySelectorAll('a[href^="magnet:"]');
  magnetLinks.forEach(el => results.push({ element: el, type:
    'magnet', source: 'href' }));

  // .torrent file links
  const torrentLinks =
    document.querySelectorAll('a[href$=".torrent"]');
  torrentLinks.forEach(el => results.push({ element: el, type:
    'torrent', source: 'href' }));

  // Data attributes that may contain magnet links
  const dataMagnetLinks = document.querySelectorAll('[data-
    magnet]');
  dataMagnetLinks.forEach(el => results.push({
    element: el,
    type: 'magnet',
    source: 'data-attribute',
    value: el.getAttribute('data-magnet')
  }));

  return results;
}

```

2.2 TreeWalker for Text Node Traversal

Claim: TreeWalker is a DOM API that provides efficient traversal of specific node types and can be faster than recursive manual DOM walking for certain operations. **Source:** MDN - TreeWalker **URL:** <https://developer.mozilla.org/en-US/docs/Web/API/TreeWalker> **Date:** N/A **Excerpt:** A TreeWalker object represents the nodes of a document subtree and a position within them. **Confidence:** high

```

/**
 * Uses TreeWalker to efficiently scan for text nodes containing
 * magnet links.
 * TreeWalker is optimized by the browser and handles edge cases
 * automatically.
 *
 * @param {Node} rootNode - Root node to scan (default:
 *   document.body)
 * @returns {Array<{textNode, parentElement, magnetUrls}>}
 */

```

```

function findMagnetLinksInTextNodes(rootNode = document.body) {
  const results = [];
  const magnetRegex = /magnet:\?xt=urn:btih:[a-f0-9]{32,40}
    [^'"\\s<>]*/gi;

  const walker = document.createTreeWalker(
    rootNode,
    NodeFilter.SHOW_TEXT,
    {
      acceptNode: (node) => {
        // Skip text nodes inside script/style tags
        const parent = node.parentElement;
        if (!parent) return NodeFilter.FILTER_SKIP;
        const parentTag = parent.tagName.toLowerCase();
        if (parentTag === 'script' || parentTag === 'style' ||
          parentTag === 'noscript') {
          return NodeFilter.FILTER_SKIP;
        }
        // Skip empty or whitespace-only nodes
        if (!node.nodeValue || !node.nodeValue.trim()) {
          return NodeFilter.FILTER_SKIP;
        }
        return NodeFilter.FILTER_ACCEPT;
      }
    },
    false
  );

  let textNode;
  while ((textNode = walker.nextNode()) !== null) {
    const matches = textNode.nodeValue.match(magnetRegex);
    if (matches && matches.length > 0) {
      results.push({
        textNode,
        parentElement: textNode.parentElement,
        magnetUrls: matches
      });
    }
  }

  return results;
}

```

2.3 Recursive DOM Traversal (Shadow DOM Compatible)

```

/**
 * Recursive DOM traversal that handles Shadow DOM boundaries.
 * More flexible than TreeWalker but requires manual
 * implementation.
 */

```

```

* @param {Node} root - Starting node
* @param {Function} callback - Called for each element
* @param {Object} options - { includeShadowDOM: boolean,
    maxDepth: number }
*/
function traverseDOM(root, callback, options = {}) {
  const { includeShadowDOM = true, maxDepth = 100 } = options;
  const visited = new Set();

  function walk(node, depth) {
    if (depth > maxDepth) return;
    if (!node) return;
    if (visited.has(node)) return; // Prevent infinite loops
    visited.add(node);

    if (node.nodeType === Node.ELEMENT_NODE) {
      callback(node);

      // Traverse children in light DOM
      if (node.children) {
        for (const child of node.children) {
          walk(child, depth + 1);
        }
      }

      // Traverse shadow DOM if present
      if (includeShadowDOM && node.shadowRoot) {
        walk(node.shadowRoot, depth + 1);
      }
    }

    // Also traverse childNodes for non-element nodes (text,
    // comments)
    if (node.childNodes) {
      for (const child of node.childNodes) {
        if (child.nodeType !== Node.ELEMENT_NODE) {
          walk(child, depth + 1);
        }
      }
    }
  }

  walk(root, 0);
}

```

3. Link Element Detection Patterns

3.1 Comprehensive Link Scanner

```
/**
 * Complete link scanner that checks all common patterns for
 * torrent/magnet links.
 * Designed to be extensible with site-specific selectors.
 */
class LinkScanner {
  constructor() {
    // Magnet URI detection regex - matches the xt=urn:btih:
    // prefix with hash
    this.magnetRegex = /magnet:\?xt=urn:btih:[a-f0-9]{32,40}
    [^'"\\s<>]*/gi;

    // Site-specific CSS selectors for known torrent sites
    this.siteSelectors = {
      'thepiratebay': {
        magnet: ['a[href^="magnet:"]', '.download
        a[href^="magnet:"]'],
        torrent: ['a[href$=".torrent"]'],
        details: { name: '#title', seeders: 'dt:first-child +
        dd' }
      },
      '1337x': {
        magnet: ['a[href^="magnet:"]', '.torrent-detail-page
        a[href^="magnet:"]'],
        torrent: ['a[href$=".torrent"]'],
        details: { name: '.box-info-heading h1' }
      },
      'nyaa': {
        magnet: ['a[href^="magnet:"]', '.torrent-download-
        link[href^="magnet:"]'],
        torrent: ['a[href$=".torrent"]']
      },
      'rutracker': {
        magnet: ['a[href^="magnet:"]', '.magnet-
        link[href^="magnet:"]'],
        torrent: ['a[href$=".torrent"]']
      }
    };
  }

  /**
   * Scan all <a> tags for magnet/torrent hrefs
   */
  scanAnchorLinks(container = document) {
    const results = [];
    const links = container.querySelectorAll('a');

    for (const link of links) {
```

```

    const href = link.getAttribute('href');
    if (!href) continue;

    // Check for magnet protocol
    if (href.startsWith('magnet:')) {
        results.push({
            element: link,
            type: 'magnet',
            url: href,
            text: link.textContent.trim(),
            title: link.getAttribute('title') || ''
        });
        continue;
    }

    // Check for .torrent file
    if (href.endsWith('.torrent') ||
        href.includes('.torrent?')) {
        results.push({
            element: link,
            type: 'torrent-file',
            url: href,
            text: link.textContent.trim()
        });
        continue;
    }

    // Check for data attributes
    const dataMagnet = link.getAttribute('data-magnet');
    if (dataMagnet) {
        results.push({
            element: link,
            type: 'magnet',
            url: dataMagnet,
            source: 'data-attribute'
        });
    }
}

return results;
}

/**
 * Check for onclick handlers that open magnet links
 */
scanClickHandlers(container = document) {
    const results = [];
    const elements = container.querySelectorAll('[onclick]');

    for (const el of elements) {
        const onclick = el.getAttribute('onclick') || '';
    }
}

```



```

    const magnetMatch = onclick.match(/magnet:\?xt=urn:btih:[a-
      f0-9]{32,40}[^'\s]*/i);
    if (magnetMatch) {
      results.push({
        element: el,
        type: 'magnet',
        url: magnetMatch[0],
        source: 'onclick-handler'
      });
    }
  }
}

return results;
}
}

```

4. Text-Based Magnet Detection

4.1 Regex Pattern for Magnet Links

Claim: A magnet link regex must match the magnet:?xt=urn:btih: prefix followed by a 32-40 character hexadecimal hash, with optional parameters (dn, tr, xl, etc.) **Source:** magnet-link-regex npm package **URL:** <https://github.com/tiaanduplessis/magnet-link-regex/blob/master/index.js> **Date:** 2018-06-04 **Excerpt:** /magnet:\?xt=urn:[a-z0-9]+:[a-z0-9]{32,40}&dn=.\&tr=.\+/gi **Confidence:** high

```

/**
 * Comprehensive regex patterns for magnet link detection.
 * Uses multiple patterns to handle different magnet link formats.
 */
const MAGNET_PATTERNS = {
  // Full magnet link with BTIH hash (hex, 32-40 chars)
  // Matches: magnet:?xt=urn:btih:HASH&dn=NAME&tr=TRACKER...
  FULL: /magnet:\?xt=urn:btih:[a-fA-F0-9]{32,40}(?:&[^\s"'<>]+)*/gi,

  // Minimal magnet link (just hash)
  // Matches: magnet:?xt=urn:btih:HASH
  MINIMAL: /magnet:\?xt=urn:btih:[a-fA-F0-9]{32,40}/gi,

  // Magnet link with any URN type (btih, sha1, ed2k, etc.)
  // Matches: magnet:?xt=urn:TYPE:HASH
  URN_GENERIC: /magnet:\?xt=urn:[a-z0-9]+:[a-zA-Z0-9]{32,40}(?:&[^\s"'<>]+)*/gi,

  // Extract BTIH hash specifically
  // Captures group 1 as the hash
  BTIH_EXTRACT: /xt=urn:btih:([a-fA-F0-9]{32,40})/i,

```

```

    // Extract display name (dn parameter)
    DN_EXTRACT: /[?&]dn=([^\&]+)/i,

    // Extract tracker URLs (tr parameters)
    TR_EXTRACT: /[?&]tr=([^\&]+)/gi
};

/**
 * Validates a magnet link string and extracts components.
 *
 * @param {string} url - The magnet URL to validate
 * @returns {Object|null} Parsed magnet info or null if invalid
 */
function parseMagnetLink(url) {
    if (!url || !url.startsWith('magnet:')) return null;

    const btihMatch = url.match(MAGNET_PATTERNS.BTIH_EXTRACT);
    if (!btihMatch) return null;

    const hash = btihMatch[1].toLowerCase();

    // Validate hash length (32 = base32, 40 = hex)
    if (hash.length !== 32 && hash.length !== 40) return null;

    const dnMatch = url.match(MAGNET_PATTERNS.DN_EXTRACT);
    const trackers = [];
    let trMatch;
    while ((trMatch = MAGNET_PATTERNS.TR_EXTRACT.exec(url)) !==
        null) {
        try { trackers.push(decodeURIComponent(trMatch[1])); }
        catch (e) { trackers.push(trMatch[1]); }
    }
    MAGNET_PATTERNS.TR_EXTRACT.lastIndex = 0; // Reset regex

    return {
        hash,
        hashType: hash.length === 40 ? 'hex' : 'base32',
        name: dnMatch ? decodeURIComponent(dnMatch[1]) : '',
        trackers,
        raw: url
    };
}

```

4.2 Text Node Scanner

```

/**
 * Scans all text nodes on the page for magnet links that aren't
 *   in <a> tags.
 * This finds magnet links displayed as plain text, in <code>,
 *   <pre>, <span>, etc.
 *

```

```

* @param {HTMLElement} container - Container to scan
* @returns {Array<{element: HTMLElement, text: string, magnets:
    Array<string>}>}
*/
function scanTextNodesForMagnets(container = document.body) {
    const results = [];
    const processedParents = new Set(); // Avoid duplicates

    const walker = document.createTreeWalker(
        container,
        NodeFilter.SHOW_TEXT,
        null,
        false
    );

    let textNode;
    while ((textNode = walker.nextNode()) !== null) {
        const parent = textNode.parentElement;
        if (!parent) continue;

        // Skip if parent is a link (already handled by
        // scanAnchorLinks)
        if (parent.tagName === 'A' &&
            parent.getAttribute('href')?.startsWith('magnet:'))
            continue;

        // Skip script/style/noscript content
        const skipTags = ['SCRIPT', 'STYLE', 'NOSCRIPT', 'TEMPLATE'];
        if (skipTags.includes(parent.tagName)) continue;

        const text = textNode.nodeValue;
        const matches = text.match(MAGNET_PATTERNS.FULL);

        if (matches && matches.length > 0) {
            if (!processedParents.has(parent)) {
                processedParents.add(parent);
                results.push({
                    element: parent,
                    text: text.trim(),
                    magnets: matches
                });
            }
        }
    }

    return results;
}

```

5. Magnet Link URI Patterns

5.1 Magnet URI Specification

Claim: The Magnet URI scheme defines specific parameters: xt (exact topic), dn (display name), tr (tracker), xl (exact length), kt (keyword topic), ws (web seed), xs (exact source), as (acceptable source), mt (manifest topic). **Source:** Baiduwiki - Magnet URI scheme **URL:** <https://baike.baidu.com/en/item/Magnet%20URI%20scheme/1415756> **Date:** N/A **Excerpt:** xt=urn:btih:[BitTorrent Info Hash (Hexadecimal)]... This is the identifier of the file and is indispensable. **Confidence:** high

5.2 Magnet URI Components Reference

Parameter	Name	Required	Description
xt	Exact Topic	Yes	URN with file hash (e.g., urn:btih:HASH)
dn	Display Name	No	Filename shown to user
tr	Tracker	No	Tracker URL for peer discovery
xl	Exact Length	No	File size in bytes
kt	Keyword Topic	No	Search keywords
ws	Web Seed	No	HTTP/FTP seed URL
xs	Exact Source	No	P2P source address
as	Acceptable Source	No	Fallback web server URL
mt	Manifest Topic	No	Link to metafile with magnet list
so	Select Only	No	Specific file indices to download

5.3 Modern Magnet Link Formats

Claim: BitTorrent v3.1 introduces new hash algorithms beyond SHA-1, with magnet links using format magnet:?xt=urn:btih-SPECIFIER:hash. **Source:** Tixati Protocol v3.1 Specification **URL:** <https://tixati.com/specs/bittorrent/v3.1> **Date:** 2025-10-07 **Excerpt:**

Example: magnet:?xt=urn:btih-sha3:hkgkj2kv6g5kjmps66ec3tqbeekuojlrlnlig3bifqmsuyxmw64ua&dn=somefilename.jpg **Confidence:** medium

6. .torrent File Link Patterns

6.1 Detection Patterns

```
/**
 * Comprehensive .torrent link detection.
 */
const TORRENT_PATTERNS = {
  // Direct .torrent file link
  DIRECT: /\.(torrent(?:\?.*)?)$/i,

  // Download endpoints that serve .torrent files
  DOWNLOAD_ENDPOINTS: [
    /\download\.php\?.*torrent/i,
    /\torrent\download\//i,
    /\download\torrent\//i,
    /\torrents\download\//i,
    /\get\torrent\//i
  ],

  // Common file hosting patterns for torrents
  FILE_HOSTS: [
    /torcache\.net/i,
    /itorrents\.org/i,
    /zoink\.it/i,
    /googleapis\.com.*\.torrent/i,
    /rarbg\.to.*\.torrent/i
  ]
};

/**
 * Check if a URL points to a .torrent file or torrent download
 * endpoint.
 */
function isTorrentUrl(url) {
  if (!url) return false;

  // Direct .torrent file
  if (TORRENT_PATTERNS.DIRECT.test(url)) return true;

  // Download endpoints
  for (const pattern of TORRENT_PATTERNS.DOWNLOAD_ENDPOINTS) {
    if (pattern.test(url)) return true;
  }
}
```

```

    // Known torrent file hosts
    for (const pattern of TORRENT_PATTERNS.FILE_HOSTS) {
        if (pattern.test(url)) return true;
    }

    return false;
}

```

7. MutationObserver for Dynamic Content

7.1 Basic MutationObserver Setup

Claim: MutationObserver fires its callback on the microtask queue, making it ~88x faster than polling with setTimeout for DOM change detection. **Source:** Use a MutationObserver to Handle DOM Nodes That Don't Exist Yet **URL:** <https://macarthur.me/posts/use-mutation-observer-to-handle-nodes-that-dont-exist-yet> **Date:** 2023-03-13 **Excerpt:** polling with a zero-delay setTimeout() ... was around 88 times slower than the MutationObserver **Confidence:** high

```

/**
 * Sets up a MutationObserver to detect dynamically added content.
 * Uses debouncing to batch rapid DOM changes and avoid excessive scanning.
 *
 * @param {Function} onChange - Callback when DOM changes are detected
 * @param {Object} options - Configuration options
 * @returns {MutationObserver} The configured observer
 */
function setupDynamicContentObserver(onChange, options = {}) {
    const {
        debounceMs = 500, // Wait for DOM changes to settle
        maxBatchSize = 100, // Max mutations to process at once
        subtree = true, // Watch entire DOM tree
        attributes = false // Watch attribute changes
        (performance cost)
    } = options;

    let debounceTimer = null;
    let pendingMutations = [];
    let isProcessing = false;

    const observer = new MutationObserver((mutations) => {
        // Filter relevant mutations (added nodes only)
        const relevantMutations = mutations.filter(m =>
            m.type === 'childList' && m.addedNodes.length > 0

```

```

);

if (relevantMutations.length === 0) return;

pendingMutations.push(...relevantMutations);

// Limit pending mutations to prevent memory issues
if (pendingMutations.length > maxBatchSize) {
  pendingMutations = pendingMutations.slice(-maxBatchSize);
}

// Debounce the scan to batch rapid changes
clearTimeout(debounceTimer);
debounceTimer = setTimeout(() => {
  if (isProcessing) return;
  isProcessing = true;

  try {
    // Collect all added nodes
    const addedNodes = [];
    for (const mutation of pendingMutations) {
      for (const node of mutation.addedNodes) {
        if (node.nodeType === Node.ELEMENT_NODE) {
          addedNodes.push(node);
        }
      }
    }
    pendingMutations = [];

    if (addedNodes.length > 0) {
      onChange(addedNodes);
    }
  } finally {
    isProcessing = false;
  }
}, debounceMs);
});

// Observe the document body for changes
observer.observe(document.body, {
  childList: true,
  subtree: subtree,
  attributes: attributes
});

return observer;
}

```

7.2 Incremental Scanning

```
/**
 * Incremental scanner that only scans newly added nodes.
 * Much more efficient than re-scanning the entire DOM.
 *
 * @param {Array<Node>} addedNodes - Nodes added by
 *   MutationObserver
 * @returns {Array<Object>} Found torrent/magnet links
 */
function incrementalScan(addedNodes) {
  const scanner = new LinkScanner();
  const results = [];

  for (const node of addedNodes) {
    // The node itself might be a matching element
    if (node.nodeType === Node.ELEMENT_NODE) {
      // Scan the element itself
      results.push(...scanner.scanAnchorLinks(node));
      results.push(...scanner.scanClickHandlers(node));

      // Scan text nodes within the element
      const textResults = scanTextNodesForMagnets(node);
      for (const tr of textResults) {
        for (const magnet of tr.magnets) {
          results.push({
            element: tr.element,
            type: 'magnet',
            url: magnet,
            source: 'text-node'
          });
        }
      }

      // Check for shadow DOM
      if (node.shadowRoot) {
        results.push(...scanner.scanAnchorLinks(node.shadowRoot));
        results.push(...scanTextNodesForMagnets(node.shadowRoot));
      }
    }
  }

  return results;
}
```

7.3 Infinite Scroll Detection

```
/**
 * Uses IntersectionObserver to detect when new content is being
 * loaded
```



```

* (infinite scroll pattern). Complements MutationObserver for
  scroll-based loading.
*
* @param {Function} onNearBottom - Called when user approaches
  page bottom
* @param {Object} options - Configuration
*/
function setupInfiniteScrollDetector(onNearBottom, options = {}) {
  const { rootMargin = '200px', threshold = 0 } = options;

  // Create a sentinel element at the bottom of the page
  let sentinel =
    document.getElementById('__torrent_scanner_sentinel');
  if (!sentinel) {
    sentinel = document.createElement('div');
    sentinel.id = '__torrent_scanner_sentinel';
    sentinel.style.cssText =
      'position:absolute;bottom:0;left:0;width:1px;height:1px;';
    document.body.appendChild(sentinel);
  }

  const observer = new IntersectionObserver((entries) => {
    if (entries[0].isIntersecting) {
      onNearBottom();
    }
  }, { rootMargin, threshold });

  observer.observe(sentinel);
  return observer;
}

```

8. iframe Handling

8.1 Cross-Origin iframe Limitations

Claim: Content scripts with `all_frames: true` are injected into all matching frames, but cross-origin iframes require special handling - content scripts can access their own DOM but cannot access the DOM of cross-origin parent/child frames from JavaScript. **Source:** MDN - content_scripts manifest **URL:** https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/manifest.json/content_scripts **Date:** 2025-11-26 **Excerpt:** "all_frames": true ... This does not inject into child frames where only their parent matches the URL requirements and the child frame does not match. **Confidence:** high

8.2 iframe Detection Strategies

```
/**
 * Strategies for handling iframes that may contain torrent links.
 * Uses multiple approaches due to cross-origin restrictions.
 */
class IFRAMEHandler {
  constructor() {
    this.knownIFrames = new Map(); // iframe -> lastSrc
  }

  /**
   * Method 1: Check if content script is running inside an
   *           iframe.
   * Use this to adapt behavior when inside a frame context.
   */
  isInsideIFrame() {
    return window.self !== window.top;
  }

  /**
   * Method 2: Try to access same-origin iframe content.
   * Only works if iframe is same-origin (limited practical use
   * for torrent sites).
   */
  tryAccessSameOriginIFrames() {
    const results = [];
    const iframes = document.querySelectorAll('iframe');

    for (const iframe of iframes) {
      try {
        // This will throw if cross-origin
        const iframeDoc = iframe.contentDocument ||
          iframe.contentWindow?.document;
        if (iframeDoc) {
          // Same-origin iframe - can scan directly
          const scanner = new LinkScanner();
          results.push(...scanner.scanAnchorLinks(iframeDoc));

          results.push(...scanner.scanTextNodesForMagnets(iframeDoc.body));
        }
      } catch (e) {
        // Cross-origin iframe - cannot access
        // Content script with all_frames:true should handle this
      }
    }

    return results;
  }

  /**
   * Method 3: Monitor iframe src changes using MutationObserver.
   */
}
```

```

* Detects when iframes are added or their source changes.
*/
monitorIframes(onIframeDetected) {
  const observer = new MutationObserver((mutations) => {
    for (const mutation of mutations) {
      for (const node of mutation.addedNodes) {
        if (node.nodeType === Node.ELEMENT_NODE) {
          // Check if the node itself is an iframe
          if (node.tagName === 'IFRAME') {
            onIframeDetected(node);
          }
          // Check if the node contains iframes
          const nestedIframes = node.querySelectorAll?.
('iframe');
          if (nestedIframes) {
            for (const iframe of nestedIframes) {
              onIframeDetected(iframe);
            }
          }
        }
      }
    }

    // Check for src attribute changes on existing iframes
    if (mutation.type === 'attributes' &&
        mutation.target.tagName === 'IFRAME' &&
        mutation.attributeName === 'src') {
      onIframeDetected(mutation.target);
    }
  });

  observer.observe(document.body, {
    childList: true,
    subtree: true,
    attributes: true,
    attributeFilter: ['src']
  });

  return observer;
}

```

8.3 Manifest Configuration for iframe Support

```

{
  "content_scripts": [
    {
      "matches": ["<all_urls>"],
      "js": ["content.js"],
      "run_at": "document_idle",
      "all_frames": true,
    }
  ]
}

```

```

    "match_about_blank": true
  }
]
}

```

Claim: match_about_blank: true allows content scripts to run in about:blank iframes, which is useful for detecting dynamically created empty iframes that get populated with content. **Source:** MDN - content_scripts manifest **URL:** https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/manifest.json/content_scripts **Date:** 2025-11-26 **Excerpt:** "match_about_blank": true ... This is especially useful to run scripts in empty iframes, whose URL is "about:blank". **Confidence:** high

9. Shadow DOM Traversal

9.1 Understanding Shadow DOM Boundaries

Claim: document.querySelectorAll() does NOT return elements inside Shadow DOM. Elements in shadow trees are encapsulated and require explicit shadow root traversal. **Source:** MDN - Using shadow DOM **URL:** https://developer.mozilla.org/en-US/docs/Web/API/Web_components/Using_shadow_DOM **Date:** 2025-12-17 **Excerpt:** document.querySelector("p") // null ... document.querySelectorAll('*') grabs all the elements in the DOM. With the shadow DOM, though, it doesn't work that way **Confidence:** high

9.2 Shadow DOM Piercing Implementation

```

/**
 * Recursively scans through Shadow DOM boundaries to find torrent
 * links.
 * Handles both open and (limited) closed shadow roots.
 *
 * @param {Node} root - Root node to scan from
 * @returns {Array<Object>} Found torrent/magnet links
 */
function scanThroughShadowDOM(root = document) {
  const results = [];
  const scanner = new LinkScanner();
  const visitedShadowRoots = new WeakSet();

  function scanNode(node) {
    if (!node) return;

    if (node.nodeType === Node.ELEMENT_NODE) {

```

```

    // Scan this element's light DOM
    results.push(...scanner.scanAnchorLinks(node));
    results.push(...scanner.scanClickHandlers(node));

    // Check for shadow root
    const shadowRoot = node.shadowRoot;
    if (shadowRoot && !visitedShadowRoots.has(shadowRoot)) {
        visitedShadowRoots.add(shadowRoot);

        // Recursively scan the shadow DOM
        scanNode(shadowRoot);
    }

    // Recurse into children
    if (node.children) {
        for (const child of node.children) {
            scanNode(child);
        }
    }
}

// Handle DocumentFragment roots (shadow roots)
if (node.nodeType === Node.DOCUMENT_FRAGMENT_NODE) {
    const children = node.children || node.querySelector?.(
        ':scope > *');
    if (children) {
        for (const child of children) {
            scanNode(child);
        }
    }
}

scanNode(root);
return results;
}

```

9.3 Using TreeWalker with Shadow DOM

```

/**
 * Alternative approach using NodeIterator to find elements with
 * shadow roots,
 * then scanning each shadow root separately.
 */
function findAllShadowHosts(root = document.body) {
    const shadowHosts = [];

    const nodeIterator = document.createNodeIterator(
        root,
        NodeFilter.SHOW_ELEMENT,
        (node) => node.shadowRoot ? NodeFilter.FILTER_ACCEPT :
            NodeFilter.FILTER_REJECT
    );
}

```

```

);

let element;
while ((element = nodeIterator.nextNode()) !== null) {
    shadowHosts.push(element);
}

return shadowHosts;
}

/**
 * Scans all shadow roots found in the document.
 */
function scanAllShadowRoots() {
    const results = [];
    const scanner = new LinkScanner();

    const shadowHosts = findAllShadowHosts();
    for (const host of shadowHosts) {
        if (host.shadowRoot) {
            results.push(...scanner.scanAnchorLinks(host.shadowRoot));
            results.push(...scanner.scanTextNodesForMagnets(host.shadowRoot));

            // Recursively check for nested shadow roots
            results.push(...scanAllShadowRoots.call(null,
                host.shadowRoot));
        }
    }

    return results;
}

```

9.4 Third-Party Library: kagekiri

Claim: kagekiri is a Salesforce library that implements `querySelectorAll` to traverse shadow DOM boundaries. **Source:** GitHub - salesforce/kagekiri **URL:** <https://github.com/salesforce/kagekiri> **Date:** 2019-08-07 **Excerpt:** Shadow DOM-piercing query APIs... kagekiri parses the CSS selector using `postcss-selector-parser`. Then it queries the entire DOM tree, traverses any shadowRoots it may find **Confidence:** high

```

// Using kagekiri for shadow DOM piercing (requires npm install
// kagekiri)
// import { querySelectorAll } from 'kagekiri';

// kagekiri.querySelectorAll('.container a[href^="magnet"]')
// would find magnet links even inside shadow DOM

```

10. Site-Specific Detection Patterns

10.1 Pattern Database

Based on analysis of popular torrent sites and userscript implementations:

```
/**
 * Site-specific selectors for known torrent sites.
 * These override generic selectors for better accuracy.
 */
const SITE_PATTERNS = {
  // The Pirate Bay patterns
  'thepiratebay': {
    domains: ['thepiratebay.org', 'thepiratebay.se',
      'piratebay.live', 'baymirror.com'],
    magnet: [
      'a[href^="magnet:"]', // Standard
      // magnet links
      '.download a[href^="magnet:"]', // Download
      // section
      'a[title^="Download this torrent using magnet"]' // TPB-
      // specific title
    ],
    torrent: [
      'a[href$=".torrent"]',
      'a[href*="/torrent/"]'
    ],
    detailsPage: {
      magnet: '.download a[href^="magnet:"]',
      name: '#title',
      seeders: 'dt:contains("Seeders") + dd'
    }
  },
  // 1337x patterns
  '1337x': {
    domains: ['1337x.to', '1337x.st', 'x1337x.ws'],
    magnet: [
      'a[href^="magnet:"]',
      '.torrent-detail-page a[href^="magnet:"]',
      'a[href*="magnet:?xt=urn:btih:"]'
    ],
    torrent: [
      'a[href$=".torrent"]',
      '.torrent-down' // Download button
    ]
  },
  // Nyaa (anime torrents)
  'nyaa': {
```

```

    domains: ['nyaa.si', 'nyaa.net'],
    magnet: [
        'a[href^="magnet:"]',
        '.torrent-download-link[href^="magnet:"]',
        'a[href*="magnet:?xt=urn:btih:"]'
    ],
    torrent: [
        'a[href$=".torrent"]'
    ]
},

// RuTracker (Russian)
'rutracker': {
    domains: ['rutracker.org', 'rutracker.net', 'rutracker.nl'],
    magnet: [
        'a[href^="magnet:"]',
        '.magnet-link[href^="magnet:"]',
        '.dl-link[href^="magnet:"]'
    ],
    torrent: [
        'a[href$=".torrent"]',
        'a.dl-stub' // Download stub
    ]
},

// General torrent indexers
'generic': {
    magnet: [
        'a[href^="magnet:"]',
        'a[href*="magnet:?xt=urn:btih:"]'
    ],
    torrent: [
        'a[href$=".torrent"]',
        'a[href*=".torrent?"]',
        'a[href*="/download/"][href*=".torrent"]'
    ]
}
};

/**
 * Detect which site pattern to use based on current domain.
 */
function detectSitePattern() {
    const hostname = window.location.hostname.toLowerCase();

    for (const [siteName, pattern] of
        Object.entries(SITE_PATTERNS)) {
        if (pattern.domains) {
            for (const domain of pattern.domains) {
                if (hostname === domain || hostname.endsWith('.' +
                    domain)) {
                    return { siteName, pattern };
                }
            }
        }
    }
}

```



```

    }
  }
}

// Default to generic pattern
return { siteName: 'generic', pattern: SITE_PATTERNS.generic };
}

```

10.2 Using Site-Specific Selectors

```

/**
 * Optimized scanner that uses site-specific selectors when
 * available.
 */
function scanWithSitePatterns(container = document) {
  const { siteName, pattern } = detectSitePattern();
  const results = [];

  // Use site-specific magnet selectors if available
  const magnetSelectors = pattern.magnet ||
    SITE_PATTERNS.generic.magnet;
  for (const selector of magnetSelectors) {
    try {
      const elements = container.querySelectorAll(selector);
      for (const el of elements) {
        const href = el.getAttribute('href');
        if (href && href.startsWith('magnet:')) {
          results.push({ element: el, type: 'magnet', url: href,
            site: siteName });
        }
      }
    } catch (e) {
      // Invalid selector, skip
    }
  }

  // Use site-specific torrent selectors
  const torrentSelectors = pattern.torrent ||
    SITE_PATTERNS.generic.torrent;
  for (const selector of torrentSelectors) {
    try {
      const elements = container.querySelectorAll(selector);
      for (const el of elements) {
        const href = el.getAttribute('href');
        if (href && isTorrentUrl(href)) {
          results.push({ element: el, type: 'torrent-file', url:
            href, site: siteName });
        }
      }
    } catch (e) {
      // Invalid selector, skip
    }
  }
}

```

```

    }
  }

  return results;
}

```

11. Performance Optimization

11.1 Debounced Scanning

Claim: Debouncing prevents excessive DOM scans by delaying execution until a burst of mutations settles, typically 300-500ms after the last mutation. **Source:** CSS-Tricks - Debouncing and Throttling Explained **URL:** <https://css-tricks.com/debouncing-throttling-explained-examples/> **Date:** 2026-02-05 **Excerpt:** Debounce: Grouping a sudden burst of events into a single one **Confidence:** high

```

/**
 * Creates a debounced function that delays invoking callback
 * until after wait milliseconds
 * have elapsed since the last time the debounced function was
 * invoked.
 */
function debounce(callback, wait = 300, options = {}) {
  const { leading = false, trailing = true, maxWait } = options;
  let timer = null;
  let lastCall = 0;
  let lastInvoke = 0;
  let lastArgs;
  let lastThis;

  function invoke(time) {
    lastInvoke = time;
    const args = lastArgs;
    const ctx = lastThis;
    lastArgs = lastThis = null;
    callback.apply(ctx, args);
  }

  function debounced(...args) {
    const now = Date.now();
    const isFirstCall = lastCall === 0;
    lastCall = now;
    lastArgs = args;
    lastThis = this;

    if (timer) clearTimeout(timer);
    if (leading && isFirstCall) invoke(now);
  }
}

```

```

    const remainingWait = maxWait
      ? Math.min(wait, maxWait - (now - lastInvoke))
      : wait;

    timer = setTimeout(() => {
      timer = null;
      if (trailing && lastArgs) invoke(Date.now());
      lastCall = 0;
    }, remainingWait);
  }

  debounced.cancel = () => {
    if (timer) clearTimeout(timer);
    timer = null;
    lastCall = 0;
    lastArgs = lastThis = null;
  };

  debounced.flush = () => {
    if (timer) {
      clearTimeout(timer);
      invoke(Date.now());
      timer = null;
    }
  };

  return debounced;
}

```

11.2 Throttled Scanning

```

/**
 * Creates a throttled function that only invokes callback at most
 * once per period.
 * Useful for scroll events and other high-frequency triggers.
 */
function throttle(callback, limit = 200) {
  let inThrottle = false;
  let trailingCall = null;

  function throttled(...args) {
    if (!inThrottle) {
      callback.apply(this, args);
      inThrottle = true;
      setTimeout(() => {
        inThrottle = false;
        if (trailingCall) {
          const { ctx, args: trailingArgs } = trailingCall;
          trailingCall = null;
          callback.apply(ctx, trailingArgs);
        }
      }, limit);
    }
  }
}

```

```

    }
    }, limit);
  } else {
    // Store trailing call to ensure last event is processed
    trailingCall = { ctx: this, args };
  }
}

throttled.cancel = () => {
  inThrottle = false;
  trailingCall = null;
};

return throttled;
}

```

11.3 Preventing Layout Thrashing

Claim: Layout thrashing occurs when JavaScript alternates between reading layout properties and writing style changes, forcing the browser to recalculate layout repeatedly. **Source:** webperf.tips - Layout Thrashing and Forced Reflows **URL:** <https://webperf.tips/tip/layout-thrashing/> **Date:** 2022-12-11 **Excerpt:** Reading positioning and styling information can be quite fast if performed on a fully cached, styled and positioned Render Tree **Confidence:** high

```

/**
 * Batch DOM reads and writes to prevent layout thrashing.
 * Separates measurement (read) from mutation (write) phases.
 */
class DOMScheduler {
  constructor() {
    this.readQueue = [];
    this.writeQueue = [];
    this.scheduled = false;
  }

  read(fn) {
    this.readQueue.push(fn);
    this.schedule();
  }

  write(fn) {
    this.writeQueue.push(fn);
    this.schedule();
  }

  schedule() {
    if (this.scheduled) return;
    this.scheduled = true;
  }
}

```

```

    requestAnimationFrame(() => {
      // Execute all reads first
      for (const fn of this.readQueue) {
        try { fn(); } catch (e) { /* ignore */ }
      }
      this.readQueue = [];

      // Execute all writes after reads
      for (const fn of this.writeQueue) {
        try { fn(); } catch (e) { /* ignore */ }
      }
      this.writeQueue = [];

      this.scheduled = false;
    });
  }
}

// Usage for marking found elements
const scheduler = new DOMScheduler();

function markElementAsFound(element) {
  scheduler.write(() => {
    element.classList.add('torrent-link-detected');
    element.setAttribute('data-torrent-found', 'true');
  });
}

```

11.4 Performance Budget

```

/**
 * Performance monitor that ensures scanning doesn't exceed time
 * budgets.
 * Yields control back to the browser if scan takes too long.
 */
class PerformanceBudget {
  constructor(maxMs = 16) { // 16ms = one frame at 60fps
    this.maxMs = maxMs;
    this.startTime = 0;
  }

  begin() {
    this.startTime = performance.now();
  }

  hasTimeRemaining() {
    return (performance.now() - this.startTime) < this.maxMs;
  }
}

/**

```

```

    * Yield control if budget is exhausted.
    * Returns a promise that resolves when work can continue.
    */
    async yieldIfNeeded() {
        if (!this.hasTimeRemaining()) {
            await new Promise(resolve =>
                requestAnimationFrame(resolve));
            this.begin();
        }
    }
}

/**
 * Scans nodes respecting performance budget.
 */
async function scanWithBudget(nodes, scanner, budget = new
    PerformanceBudget(16)) {
    const results = [];
    budget.begin();

    for (const node of nodes) {
        results.push(...scanner.scanAnchorLinks(node));
        await budget.yieldIfNeeded();

        results.push(...scanner.scanClickHandlers(node));
        await budget.yieldIfNeeded();
    }

    return results;
}

```

12. Complete Implementation

12.1 Master Scanner Class

```

/**
 * Complete torrent/magnet link scanner for browser extension
 * content scripts.
 * Handles static DOM, dynamic content, iframes, and Shadow DOM.
 *
 * Usage:
 *   const scanner = new TorrentDOMScanner();
 *   scanner.start();
 *   scanner.on('found', (links) => console.log('Found:', links));
 */
class TorrentDOMScanner extends EventTarget {
    constructor(options = {}) {
        super();
    }
}

```

```

    this.options = {
        debounceMs: 500,
        scanBudgetMs: 16,
        enableMutationObserver: true,
        enableShadowDOMScan: true,
        enableTextNodeScan: true,
        enableInfiniteScroll: true,
        ...options
    };

    this.mutationObserver = null;
    this.iframeObserver = null;
    this.infiniteScrollObserver = null;
    this.linkScanner = new LinkScanner();
    this.foundLinks = new Map(); // URL -> metadata
                                (deduplication)
    this.isRunning = false;
    this.scheduler = new DOMScheduler();
}

/**
 * Start scanning the page.
 */
start() {
    if (this.isRunning) return;
    this.isRunning = true;

    // Initial full scan
    this.performFullScan();

    // Set up MutationObserver for dynamic content
    if (this.options.enableMutationObserver) {
        this.setupMutationObserver();
    }

    // Set up infinite scroll detection
    if (this.options.enableInfiniteScroll) {
        this.setupInfiniteScrollDetection();
    }
}

/**
 * Stop scanning.
 */
stop() {
    this.isRunning = false;

    if (this.mutationObserver) {
        this.mutationObserver.disconnect();
        this.mutationObserver = null;
    }
}

```

```

    if (this.infiniteScrollObserver) {
        this.infiniteScrollObserver.disconnect();
        this.infiniteScrollObserver = null;
    }

    if (this.iframeObserver) {
        this.iframeObserver.disconnect();
        this.iframeObserver = null;
    }
}

/**
 * Perform a complete scan of the current DOM.
 */
performFullScan() {
    const results = [];

    // 1. Scan with site-specific patterns
    results.push(...scanWithSitePatterns(document));

    // 2. Generic anchor link scan (fallback)
    results.push(...this.linkScanner.scanAnchorLinks(document));

    // 3. Click handler scan
    results.push(...this.linkScanner.scanClickHandlers(document));

    // 4. Text node scan for plain-text magnet links
    if (this.options.enableTextNodeScan) {
        const textResults = scanTextNodesForMagnets(document.body);
        for (const tr of textResults) {
            for (const magnet of tr.magnets) {
                results.push({
                    element: tr.element,
                    type: 'magnet',
                    url: magnet,
                    source: 'text-node'
                });
            }
        }
    }

    // 5. Shadow DOM scan
    if (this.options.enableShadowDOMScan) {
        results.push(...scanThroughShadowDOM(document));
    }

    this.processResults(results);
}

/**
 * Set up MutationObserver for incremental scanning.
 */

```



```

setupMutationObserver() {
  const debouncedIncrementalScan = debounce((addedNodes) => {
    const results = incrementalScan(addedNodes);
    this.processResults(results);
  }, this.options.debounceMs);

  this.mutationObserver =
    setupDynamicContentObserver((addedNodes) => {
      debouncedIncrementalScan(addedNodes);
    }, { debounceMs: this.options.debounceMs });
}

/**
 * Set up infinite scroll detection.
 */
setupInfiniteScrollDetection() {
  this.infiniteScrollObserver = setupInfiniteScrollDetector(()
    => {
      // Force a full scan when approaching bottom (new content
      // likely loaded)
      this.performFullScan();
    });
}

/**
 * Process scan results, deduplicate, and emit events.
 */
processResults(results) {
  const newLinks = [];

  for (const result of results) {
    const url = result.url || result.element?.getAttribute?.(
      'href');
    if (!url) continue;

    // Deduplicate
    if (this.foundLinks.has(url)) continue;

    const parsed = parseMagnetLink(url);
    const metadata = {
      url,
      type: result.type,
      source: result.source || 'dom-scan',
      site: result.site,
      element: result.element,
      parsed: parsed,
      timestamp: Date.now()
    };

    this.foundLinks.set(url, metadata);
    newLinks.push(metadata);
  }
}

```

```

    if (newLinks.length > 0) {
        this.dispatchEvent(new CustomEvent('found', { detail:
            newLinks }));
    }
}

/**
 * Get all found links so far.
 */
getAllLinks() {
    return Array.from(this.foundLinks.values());
}

/**
 * Clear all found links (useful for page navigation within
    SPA).
 */
clear() {
    this.foundLinks.clear();
}
}

```

12.2 Content Script Entry Point

```

/**
 * content.js - Content script entry point for browser extension.
 * This runs in the ISOLATED world with access to chrome.* APIs.
 */

(function() {
    'use strict';

    // Prevent double-injection
    if (window.__TORRENT_SCANNER_INITIALIZED) return;
    window.__TORRENT_SCANNER_INITIALIZED = true;

    let scanner = null;

    /**
     * Initialize the scanner when DOM is ready.
     */
    function init() {
        scanner = new TorrentDOMScanner({
            debounceMs: 500,
            enableMutationObserver: true,
            enableShadowDOMScan: true,
            enableTextNodeScan: true,
            enableInfiniteScroll: true
        });
    }

```

```

// Handle found links
scanner.addEventListener('found', (event) => {
  const links = event.detail;

  // Send to background script
  chrome.runtime.sendMessage({
    action: 'torrentLinksFound',
    data: links.map(l => ({
      url: l.url,
      type: l.type,
      source: l.source,
      hash: l.parsed?.hash,
      name: l.parsed?.name
    })))
  }).catch(err => {
    // Extension may not be listening yet
  });

  // Optional: highlight found links on page
  highlightLinks(links);
});

scanner.start();

// Listen for messages from popup/background
chrome.runtime.onMessage.addListener((message, sender,
  sendResponse) => {
  if (message.action === 'getFoundLinks') {
    sendResponse({ links: scanner.getAllLinks() });
    return true;
  }
  if (message.action === 'rescan') {
    scanner.clear();
    scanner.performFullScan();
    sendResponse({ status: 'rescanned' });
    return true;
  }
});
}

/**
 * Highlight found torrent links on the page (optional visual
 * feedback).
 */
function highlightLinks(links) {
  for (const link of links) {
    if (link.element && link.element.style) {
      link.element.style.outline = '2px solid #4CAF50';
      link.element.style.outlineOffset = '2px';
    }
  }
}
}

```

```

// Initialize when DOM is ready
if (document.readyState === 'loading') {
    document.addEventListener('DOMContentLoaded', init);
} else {
    init();
}

})();

```

12.3 Manifest V3 Configuration

```

{
  "manifest_version": 3,
  "name": "Torrent Link Scanner",
  "version": "1.0.0",
  "description": "Detects torrent and magnet links on web pages",
  "permissions": [
    "activeTab",
    "scripting"
  ],
  "host_permissions": [
    "<all_urls>"
  ],
  "content_scripts": [
    {
      "matches": ["<all_urls>"],
      "js": ["content.js"],
      "run_at": "document_idle",
      "all_frames": true,
      "match_about_blank": true
    }
  ],
  "web_accessible_resources": [
    {
      "resources": ["inject.js"],
      "matches": ["<all_urls>"]
    }
  ],
  "background": {
    "service_worker": "background.js"
  },
  "action": {
    "default_popup": "popup.html",
    "default_icon": {
      "16": "icon16.png",
      "48": "icon48.png",
      "128": "icon128.png"
    }
  }
}

```

```
}  
}
```

13. Edge Cases and Error Handling

13.1 Common Edge Cases

```
/**  
 * Edge case handlers for robust scanning.  
 */  
const EDGE_CASE_HANDLERS = {  
  /**  
   * Handle encoded magnet URLs in href.  
   * Some sites URL-encode magnet links.  
   */  
  decodeEncodedMagnet(element) {  
    const href = element.getAttribute('href');  
    if (!href) return null;  
  
    // Check if the href is URL-encoded  
    if (href.includes('%3A') || href.includes('%3F')) {  
      try {  
        const decoded = decodeURIComponent(href);  
        if (decoded.startsWith('magnet:')) {  
          return decoded;  
        }  
      } catch (e) {  
        // Invalid encoding, ignore  
      }  
    }  
    return null;  
  },  
  
  /**  
   * Handle magnet links in javascript: URLs.  
   * e.g., href="javascript:window.location='magnet:?xt=...'"  
   */  
  extractFromJavaScriptUrl(element) {  
    const href = element.getAttribute('href') || '';  
    const jsMatch = href.match(/javascript:.*?(magnet:\?  
      xt=urn:btih:[a-f0-9]{32,40}[^"'\s]*)/i);  
    return jsMatch ? jsMatch[1] : null;  
  },  
  
  /**  
   * Handle Base32-encoded BTIH hashes (32 chars).  
   * Normal hex hashes are 40 chars.  
   */  
  normalizeHash(hash) {
```

```

    if (hash.length === 40) {
        return hash.toLowerCase(); // Hex hash
    }
    if (hash.length === 32) {
        return hash.toUpperCase(); // Base32 hash
    }
    return hash;
},

/**
 * Handle links that use data-* attributes instead of href.
 */
extractFromDataAttributes(element) {
    for (const attr of element.attributes) {
        if (attr.name.startsWith('data-') &&
            attr.value.startsWith('magnet:')) {
            return attr.value;
        }
        // Some sites store the hash only
        if (attr.name === 'data-hash' && attr.value.length === 40) {
            return `magnet:?xt=urn:btih:${attr.value}`;
        }
    }
    return null;
},

/**
 * Prevent duplicate detection across multiple scan passes.
 */
deduplicateKey(url) {
    // Normalize URL for deduplication
    try {
        const parsed = parseMagnetLink(url);
        if (parsed) {
            return `magnet:${parsed.hash}`;
        }
    } catch (e) {}
    return url;
}
};

```

13.2 Error Handling

```

/**
 * Wraps any function with error handling that won't break the
 * scanner.
 */
function safeExecute(fn, fallback = null, context = 'unknown') {
    try {
        return fn();
    } catch (error) {

```

```

        console.warn(`[TorrentScanner] Error in ${context}:`, error);
        return fallback;
    }
}

/**
 * Safe querySelectorAll that won't throw on invalid selectors.
 */
function safeQuerySelectorAll(container, selector) {
    return safeExecute(
        () => container.querySelectorAll(selector),
        [],
        `querySelectorAll("${selector}")`
    );
}

/**
 * Safe TreeWalker creation.
 */
function safeCreateTreeWalker(root, whatToShow, filter) {
    return safeExecute(
        () => document.createTreeWalker(root, whatToShow, filter,
            false),
        null,
        'createTreeWalker'
    );
}

```

13.3 Memory Leak Prevention

```

/**
 * Cleanup utilities to prevent memory leaks in long-running
 * content scripts.
 */
class ScannerCleanup {
    constructor() {
        this.observers = [];
        this.timers = [];
        this.listeners = [];
    }

    trackObserver(observer) {
        this.observers.push(observer);
        return observer;
    }

    trackTimer(timer) {
        this.timers.push(timer);
        return timer;
    }
}

```

```

trackListener(target, type, handler) {
  target.addEventListener(type, handler);
  this.listeners.push({ target, type, handler });
}

/**
 * Call this when the scanner is being destroyed or page is
 * navigating.
 */
cleanupAll() {
  for (const observer of this.observers) {
    try { observer.disconnect(); } catch (e) {}
  }
  this.observers = [];

  for (const timer of this.timers) {
    try { clearTimeout(timer); clearInterval(timer); }
    catch (e) {}
  }
  this.timers = [];

  for (const { target, type, handler } of this.listeners) {
    try { target.removeEventListener(type, handler); }
    catch (e) {}
  }
  this.listeners = [];
}
}

```

14. Test Cases

14.1 HTML Test Fixtures

```

<!-- Test Case 1: Basic magnet link in <a> tag -->
<a href="magnet:?
  xt=urn:btih:211361b71d2e589ca44b99e0b9ce7d838d58e48a&dn=Ubuntu&tr=udp://
  tracker.example.com">Download Ubuntu</a>

<!-- Test Case 2: Magnet link as plain text -->
<code>magnet:?
  xt=urn:btih:42f7d12bdf685907ddc7eae532d3e4214e8f12d5&dn=Example</
  code>

<!-- Test Case 3: .torrent file link -->
<a href="https://example.com/file.torrent">Download .torrent</a>

<!-- Test Case 4: Data attribute magnet -->
<button data-magnet="magnet:?
  xt=urn:btih:37a8e3b4564996c13b408ebb695431dbf9f0e1c8">Copy
  Magnet</button>

```


[illegible]

14.2 JavaScript Test Suite

```
/**
 * Test suite for the torrent scanner.
 * Can be run in browser console or with a test framework.
 */
function runTests() {
  let passed = 0;
  let failed = 0;

  function assert(condition, name) {
    if (condition) { passed++; console.log(`PASS: ${name}`); }
    else { failed++; console.error(`FAIL: ${name}`); }
  }

  // Test parseMagnetLink
  const validMagnet = 'magnet:?'
    + 'xt=urn:btih:211361b71d2e589ca44b99e0b9ce7d838d58e48a&dn=Ubuntu&tr=udp://'
    + 'tracker.example.com';
  const parsed = parseMagnetLink(validMagnet);
```

[illegible]

15. Citations and Sources

Primary Sources

#	Source	URL	Date	Authority
1	MDN - MutationObserver	https://developer.mozilla.org/en-US/docs/Web/API/MutationObserver	2025-06-10	S (MDN)
2	MDN - content_scripts manifest	https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/manifest.json/content_scripts	2025-11-26	S (MDN)
3	MDN - Using shadow DOM	https://developer.mozilla.org/en-US/docs/Web/API/Web_components/Using_shadow_DOM	2025-12-17	S (MDN)
4	Chrome for Developers - Content Scripts	https://developer.chrome.com/docs/extensions/develop/concepts/content-scripts	2012-09-17	S (Official)
5	Chrome - scripting API	https://developer.chrome.com/docs/extensions/reference/api/scripting	2026-01-08	S (Official)
6	MS Edge - Minimize page load time	https://learn.microsoft.com/en-us/microsoft-edge/extensions/developer-guide/minimize-page-load-time-impact	2024-09-18	S (Official)
7	W3C - DOM Traversal Spec	https://www.w3.org/TR/DOM-Level-2-Traversal-Range/traversal.html	N/A	S (W3C)
8	CSS-Tricks - Debouncing/Throttling	https://css-tricks.com/debouncing-throttling-explained-examples/	2026-02-05	A (Authority)
9	webperf.tips - Layout Thrashing	https://webperf.tips/tip/layout-thrashing/	2022-12-11	A (Authority)
10			N/A	S (Spec)

#	Source	URL	Date	Authority
11	BEP 0053 - Magnet URI extension	https://www.bittorrent.org/beps/bep_0053.html	2025-10-07	A (Spec)
	Tixati Protocol v3.1	https://tixati.com/specs/bittorrent/v3.1		

Implementation References

#	Source	URL	Date	Authority
12	magnet-link-regex npm	https://github.com/tiaanduplessis/magnet-link-regex	2018-06-04	S (GitHub)
13	parse-torrent (WebTorrent)	https://github.com/webtorrent/parse-torrent	2014-02-24	S (GitHub)
14	magnet-uri (WebTorrent)	https://github.com/webtorrent/magnet-uri	2013-10-28	S (GitHub)
15	kagekiri (Salesforce)	https://github.com/salesforce/kagekiri	2019-08-07	S (GitHub)
16	Magnet2Deluge extension	https://github.com/jkctech/Magnet2Deluge	2025-07-06	S (GitHub)
17	magnet-linker-browser-extension	https://github.com/trossr32/magnet-linker-browser-extension	2019-07-03	S (GitHub)
18	Torrent Search extension	https://webextension.org/listing/torrent-search.html	2024-08-04	A (Product)
19	Magnet Link Finder userscript	https://gist.github.com/patik/8388768	2014-01-12	S (GitHub Gist)
20	MutationObserver guide	https://macarthur.me/posts/use-mutation-observer-to-	2023-03-13	A (Blog)

#	Source	URL	Date	Authority
		handle-nodes-that-dont-exist-yet		

Summary

Key Technical Decisions

1. **Use `document_idle` with DOM readiness checks** - The default timing is best for most cases, with fallback checks for `readyState`.
2. **Combine CSS selectors + TreeWalker + MutationObserver** - CSS selectors are fastest for known patterns, TreeWalker handles text nodes, MutationObserver catches dynamic content.
3. **Debounced incremental scanning (500ms)** - Batches rapid DOM changes to avoid excessive CPU usage while remaining responsive.
4. **Performance budget (16ms/frame)** - Yields control back to the browser to prevent UI jank on heavy pages.
5. **Deduplication by info hash** - Normalizes and deduplicates magnet links by their BTIH hash, not full URL.
6. **`all_frames: true` + `match_about_blank: true`** - Ensures scanning inside iframes and dynamically created blank frames.
7. **Shadow DOM recursive traversal** - Manual recursion into `shadowRoot` properties since `querySelectorAll` doesn't pierce shadow boundaries.

Browser Compatibility

Feature	Chrome	Firefox	Safari	Edge
MutationObserver	Yes (26+)	Yes (14+)	Yes (7+)	Yes (12+)
IntersectionObserver	Yes (51+)	Yes (55+)	Yes (12.1+)	Yes (15+)
Shadow DOM v1	Yes (53+)	Yes (63+)	Yes (10.1+)	Yes (79+)
TreeWalker	All	All	All	All
Manifest V3	Yes		Yes	Yes

Feature	Chrome	Firefox	Safari	Edge
		Yes (101+)		

*Document generated: 2025-01-15 Research dimension: DOM
Scraping & Dynamic Content Detection Steps used: 13/60*